

In [5]: `# MOVIE RATING ANALYTICS (ADVANCED VISULIZATION)`

```
import pandas as pd
import os
```

In [6]: `os.getcwd()` *# if you want to change the working directory*

Out[6]: 'c:\\Users\\mahes\\OneDrive - United Nations\\Data Science course Naresh\\3 Data Science\\September\\3 Sep\\MOVIE RATINGS _ ADVANCE VISUALIZATION _ EDA 1\\MOVIE RATINGS _ ADVANCE VISUALIZATION _ EDA 1'

In [7]: `movies = pd.read_csv(r"C:\Users\mahes\OneDrive - United Nations\Data Science course`

In [8]: `movies`

Out[8]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

In [9]: `len(movies)`

Out[9]: 559

In [10]: `movies.head()`

Out[10]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [11]: `movies.tail()`

Out[11]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

In [12]: `movies.columns`

Out[12]: `Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %', 'Budget (million $)', 'Year of release'], dtype='object')`

In [17]: `movies.columns = ['Film', 'Genre', 'CriticRating', 'AudienceRating', 'BudgetMillions`

In [18]: `movies.head() # Removed spaces & % removed noise characters`

Out[18]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [19]: `movies.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Film                  559 non-null   object
1   Genre                  559 non-null   object
2   CriticRating           559 non-null   int64
3   AudienceRating         559 non-null   int64
4   BudgetMillions         559 non-null   int64
5   Year                   559 non-null   int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

```
In [20]: movies.describe()
# if you look at the year the data type is int but when you look at the mean value
# we have to change to category type
# also from object datatype we will convert to category datatypes
#
```

Out[20]:

	CriticRating	AudienceRating	BudgetMillions	Year
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

```
In [21]: movies['Film']
#movies['Audience Ratings %']
```

Out[21]:

0	(500) Days of Summer
1	10,000 B.C.
2	12 Rounds
3	127 Hours
4	17 Again
...	
554	Your Highness
555	Youth in Revolt
556	Zodiac
557	Zombieland
558	Zookeeper

Name: Film, Length: 559, dtype: object

```
In [22]: movies.Film
```

```
Out[22]: 0      (500) Days of Summer
         1      10,000 B.C.
         2      12 Rounds
         3      127 Hours
         4      17 Again
         ...
        554      Your Highness
        555      Youth in Revolt
        556      Zodiac
        557      Zombieland
        558      Zookeeper
        Name: Film, Length: 559, dtype: object
```

```
In [23]: movies.Film = movies.Film.astype('category')
```

```
In [24]: movies.Film
```

```
Out[24]: 0      (500) Days of Summer
         1      10,000 B.C.
         2      12 Rounds
         3      127 Hours
         4      17 Again
         ...
        554      Your Highness
        555      Youth in Revolt
        556      Zodiac
        557      Zombieland
        558      Zookeeper
        Name: Film, Length: 559, dtype: category
        Categories (559, object): ['(500) Days of Summer ', '10,000 B.C.', '12 Rounds ',
        '127 Hours', ..., 'Youth in Revolt', 'Zodiac', 'Zombieland ', 'Zookeeper']
```

```
In [26]: movies.head()
```

```
Out[26]:
```

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

```
In [27]: movies.info()

# now the same thing we will change genra to category & year to category
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Film                  559 non-null   category
1   Genre                 559 non-null   object
2   CriticRating          559 non-null   int64
3   AudienceRating        559 non-null   int64
4   BudgetMillions        559 non-null   int64
5   Year                  559 non-null   int64
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

```
In [28]: movies.Genre = movies.Genre.astype('category')
movies.Year = movies.Year.astype('category')
```

```
In [29]: movies.Genre
```

```
Out[29]: 0      Comedy
1      Adventure
2      Action
3      Adventure
4      Comedy
...
554    Comedy
555    Comedy
556    Thriller
557    Action
558    Comedy
Name: Genre, Length: 559, dtype: category
Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance', 'Thriller']
```

```
In [30]: movies.Year # is it real no. year you can take average,min,max but out come have no
```

```
Out[30]: 0      2009
1      2008
2      2009
3      2010
4      2009
...
554    2011
555    2009
556    2007
557    2009
558    2011
Name: Year, Length: 559, dtype: category
Categories (5, int64): [2007, 2008, 2009, 2010, 2011]
```

```
In [31]: movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Film                  559 non-null   category
1   Genre                 559 non-null   category
2   CriticRating          559 non-null   int64
3   AudienceRating        559 non-null   int64
4   BudgetMillions        559 non-null   int64
5   Year                  559 non-null   category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

```
In [32]: movies.Genre.cat.categories
```

```
Out[32]: Index(['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance',
               'Thriller'],
              dtype='object')
```

```
In [33]: movies.describe()
#now when you see the descript you will get only integer value mean, standard devia
```

```
Out[33]:
```

	CriticRating	AudienceRating	BudgetMillions
count	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136
std	26.413091	16.826887	48.731817
min	0.000000	0.000000	0.000000
25%	25.000000	47.000000	20.000000
50%	46.000000	58.000000	35.000000
75%	70.000000	72.000000	65.000000
max	97.000000	96.000000	300.000000

```
In [75]: # How to working with joint plots

from matplotlib import pyplot as plt
import seaborn as sns
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

- basically joint plot is a scatter plot & it find the relation b/w audience & critics
- also if you look up you can find the uniform distribution (critics) and normal distribution (audience)

```
In [35]: j = sns.jointplot( data = movies, x = 'CriticRating', y = 'AudienceRating')
```

```
# Audience rating is more dominant then critics rating
# Based on this we find out as most people are most liklihood to watch audience rat
# Let me explain the excel - if you filter audience rating & critic rating. critic
```

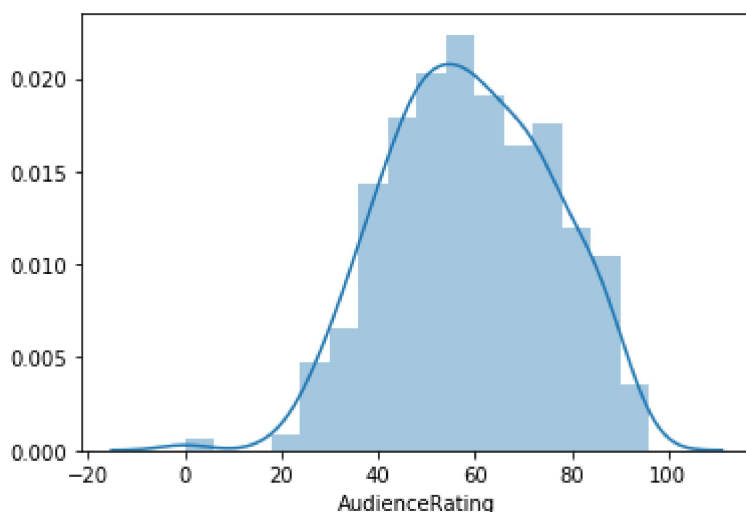
```
In [36]: j = sns.jointplot( data = movies, x = 'CriticRating', y = 'AudienceRating', kind='h'
#j = sns.jointplot( data = movies, x = 'CriticRating', y = 'AudienceRating', kind='
```

```
In [28]: #Histograms

# <<< chat1

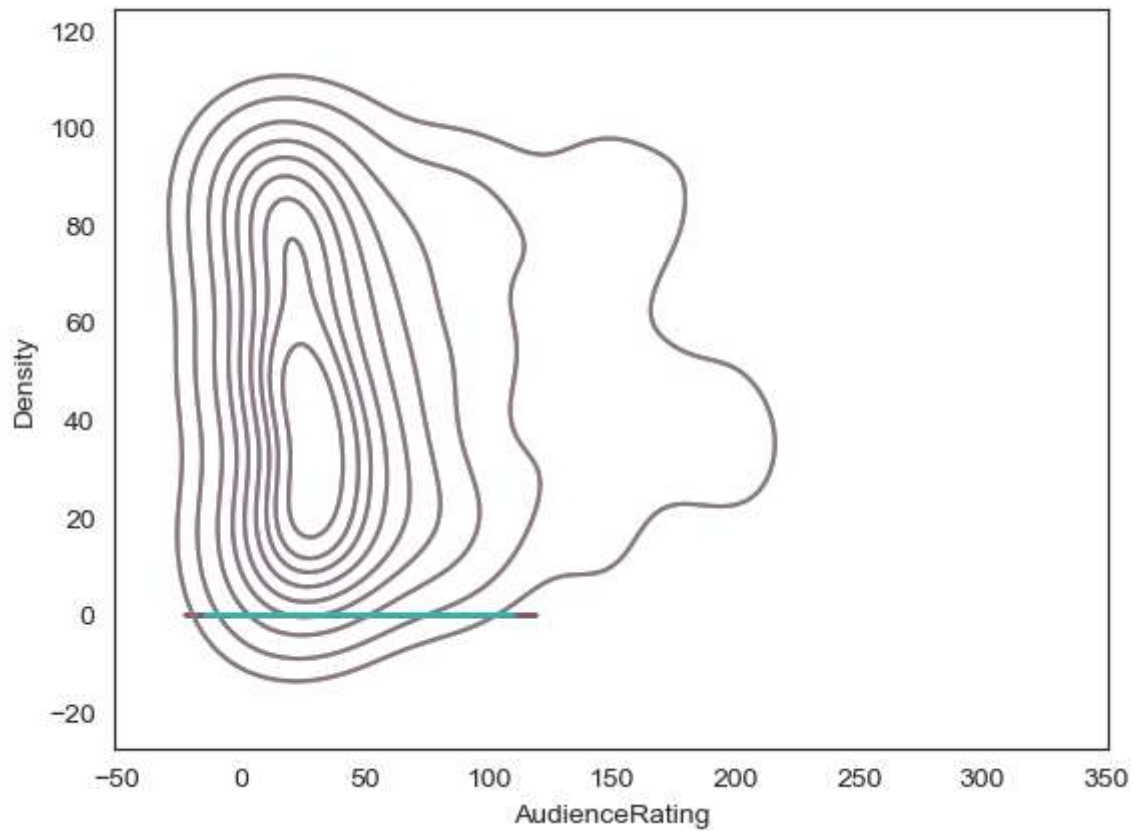
m1 = sns.distplot(movies.AudienceRating)

#y - axis generated by seaborn automatically that is the powefull of seaborn galler
```

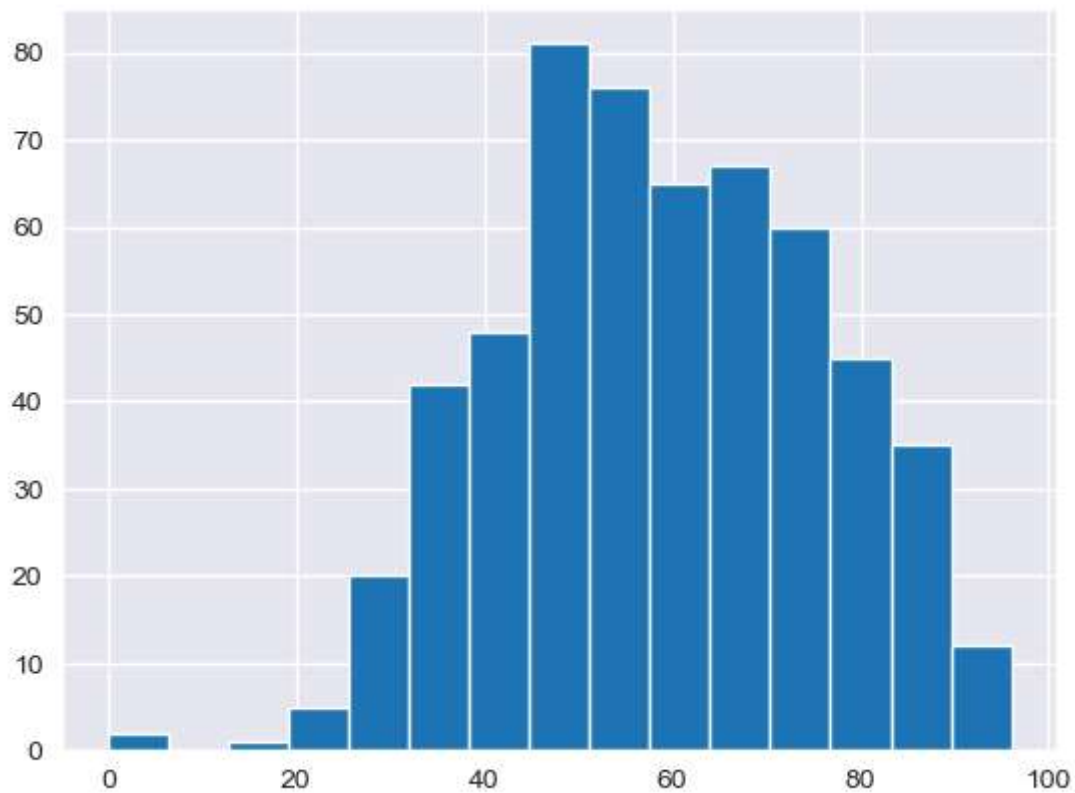


```
In [ ]: sns.set_style('darkgrid')
```

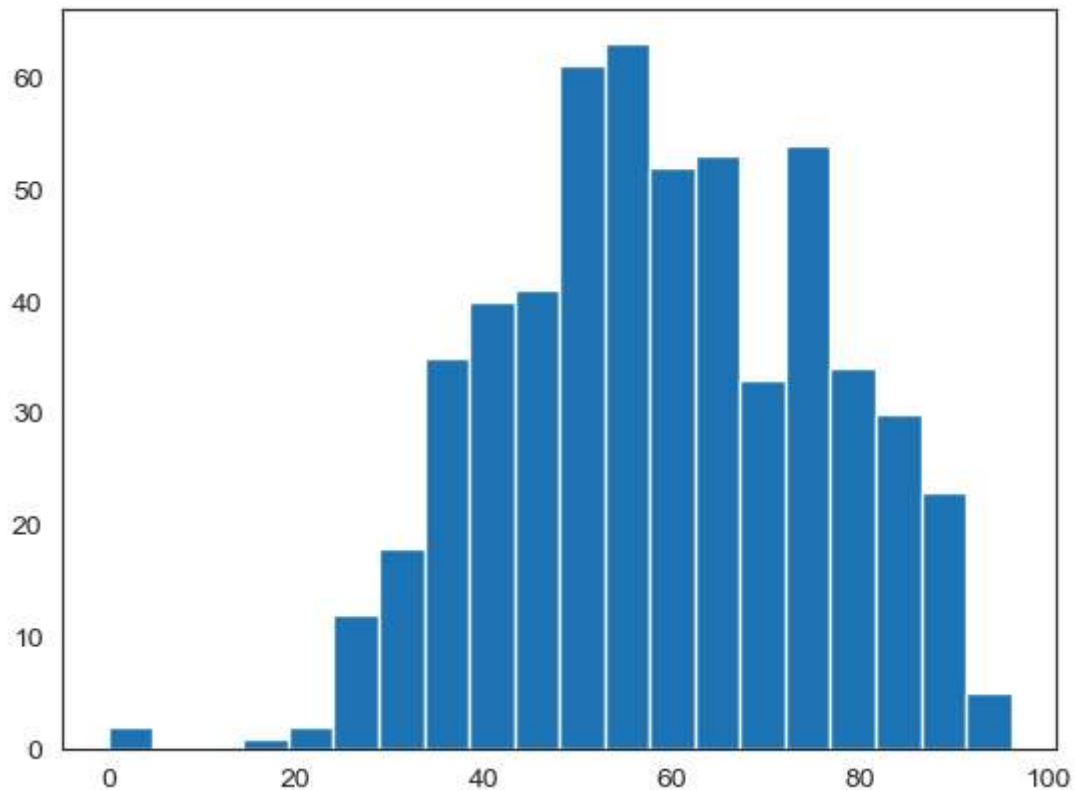
```
In [78]: m2 = sns.distplot(movies.AudienceRating, bins = 15)
plt.show()
```



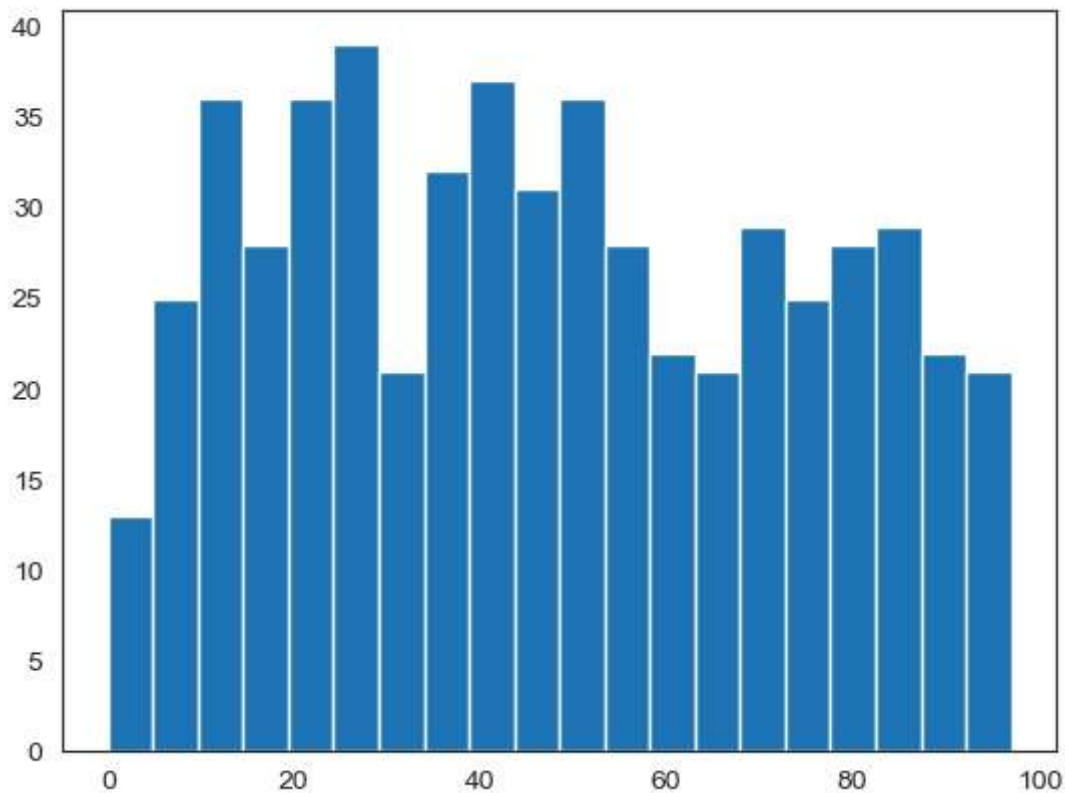
```
In [80]: #sns.set_style('darkgrid')
n1 = plt.hist(movies.AudienceRating, bins=15)
plt.show()
```




```
In [81]: sns.set_style('white') #normal distribution & called as bell curve  
n1 = plt.hist(movies.AudienceRating, bins=20)  
plt.show()
```



```
In [82]: n1 = plt.hist(movies.CriticRating, bins=20) #uniform distribution  
plt.show()
```

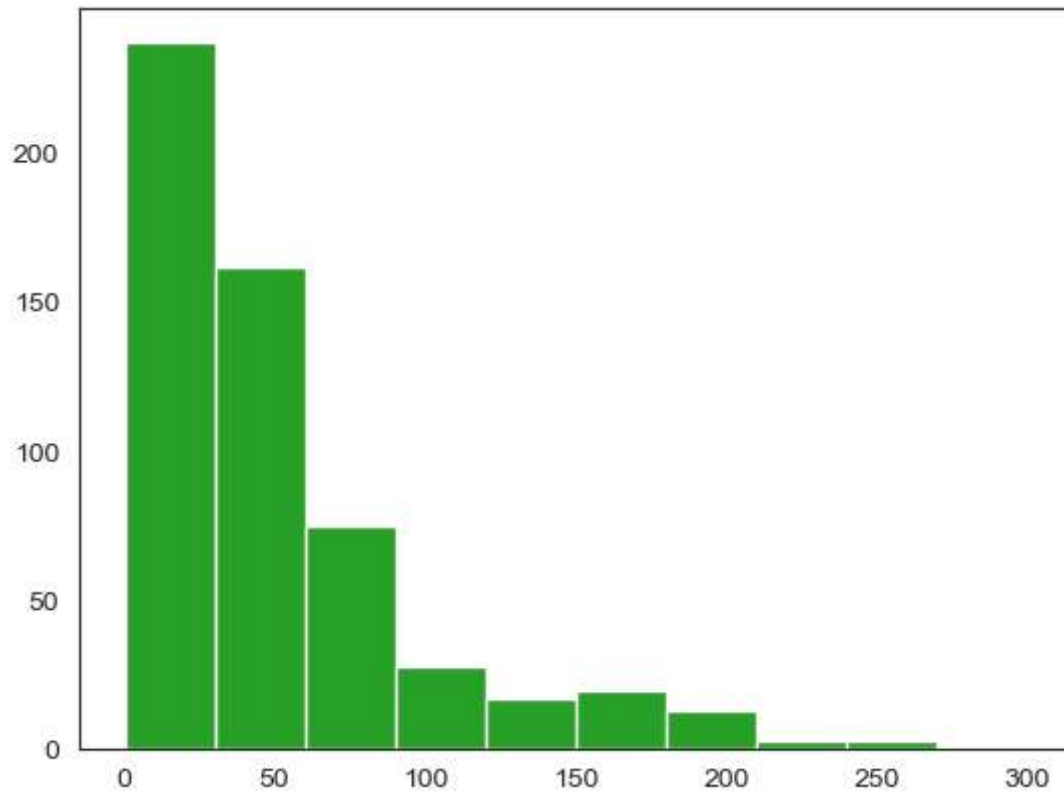


```
In [42]: # <<< chat - 2
```

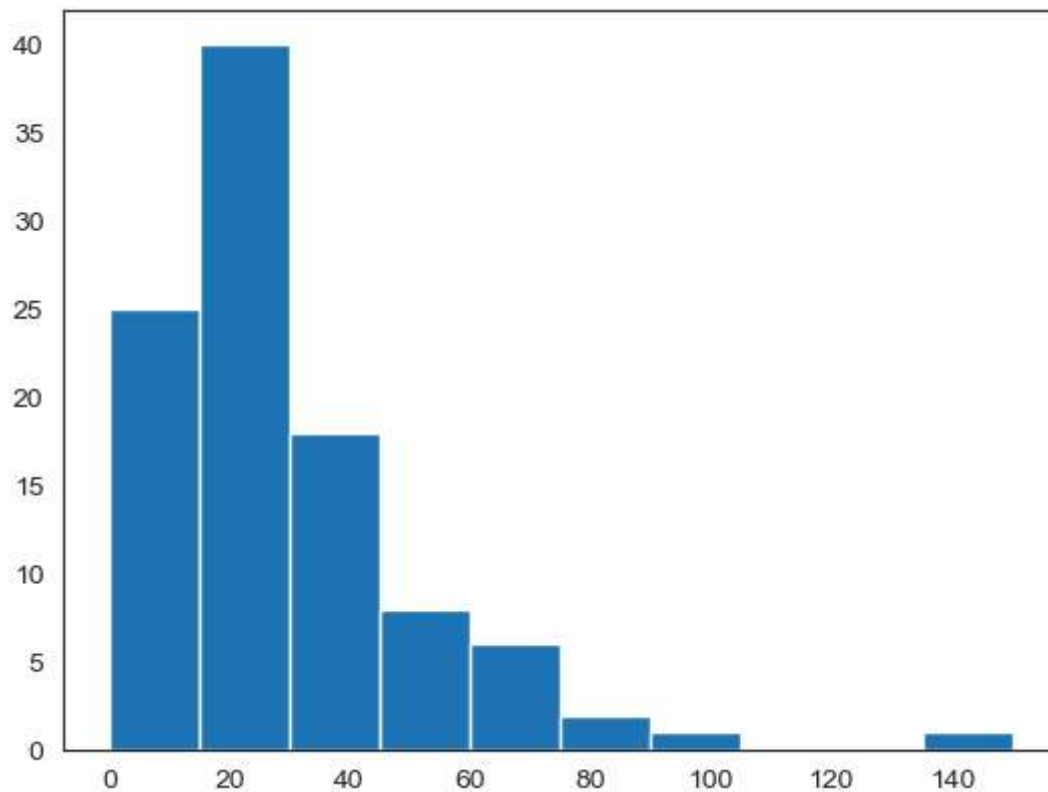
```
# Creating stacked histograms & this is bit tough to understand
```

```
In [86]: #h1 = plt.hist(movies.BudgetMillions)
```

```
plt.hist(movies.BudgetMillions)  
plt.show()
```



```
In [87]: plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions)
plt.show()
```



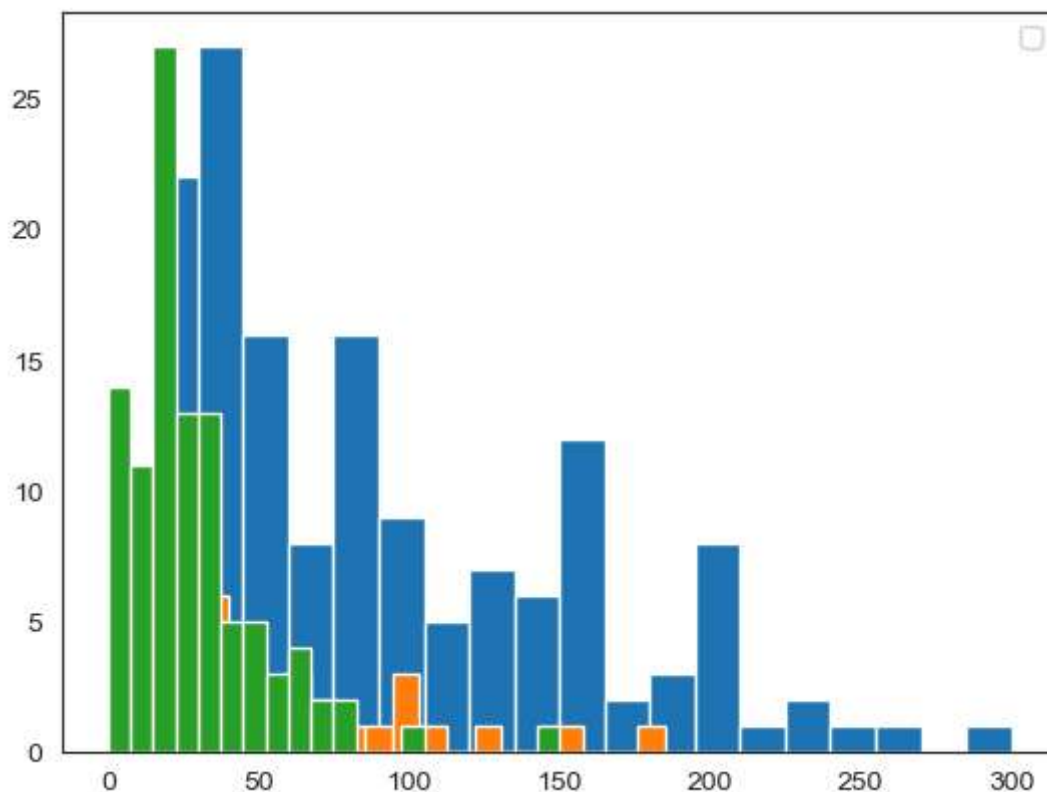
```
In [88]: movies.head()
```

Out[88]:

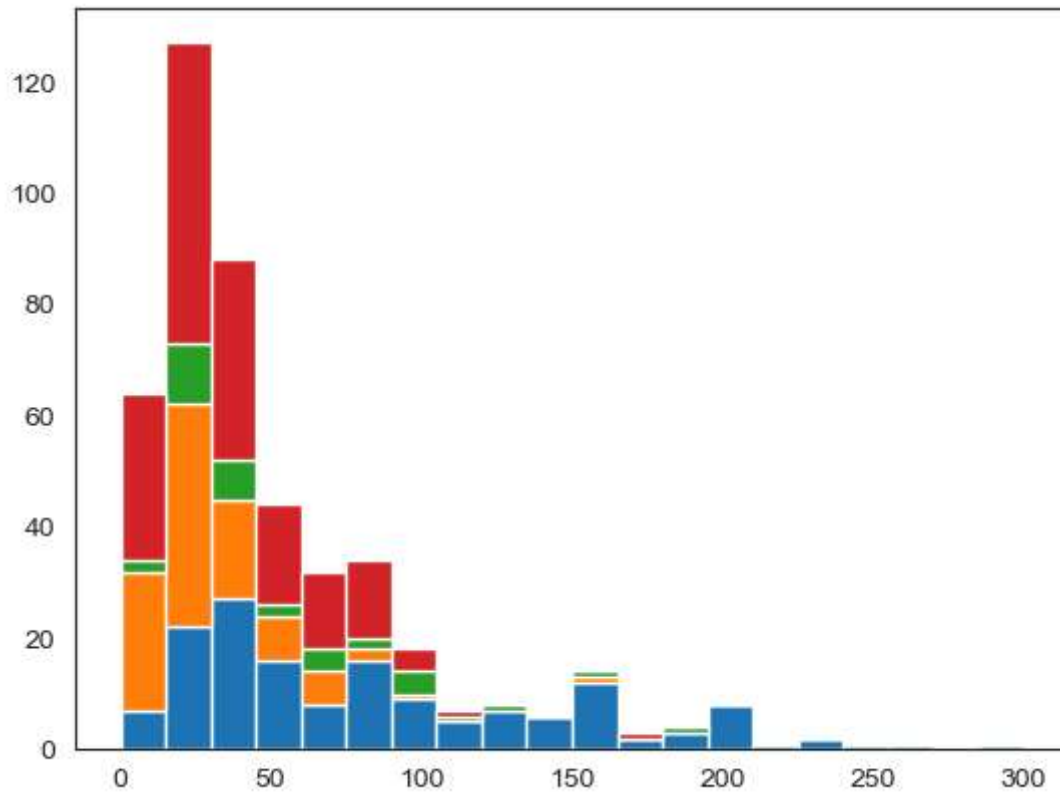
	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [89]: `#movies.Genre.unique()`In [90]: `# Below plots are stacked histogram becuse overlaped`

```
plt.hist(movies[movies.Genre == 'Action'].BudgetMillions, bins = 20)
plt.hist(movies[movies.Genre == 'Thriller'].BudgetMillions, bins = 20)
plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions, bins = 20)
plt.legend()
plt.show()
```



```
In [91]: plt.hist([movies[movies.Genre == 'Action'].BudgetMillions, \
    movies[movies.Genre == 'Drama'].BudgetMillions, \
    movies[movies.Genre == 'Thriller'].BudgetMillions, \
    movies[movies.Genre == 'Comedy'].BudgetMillions], \
    bins = 20, stacked = True)
plt.show()
```

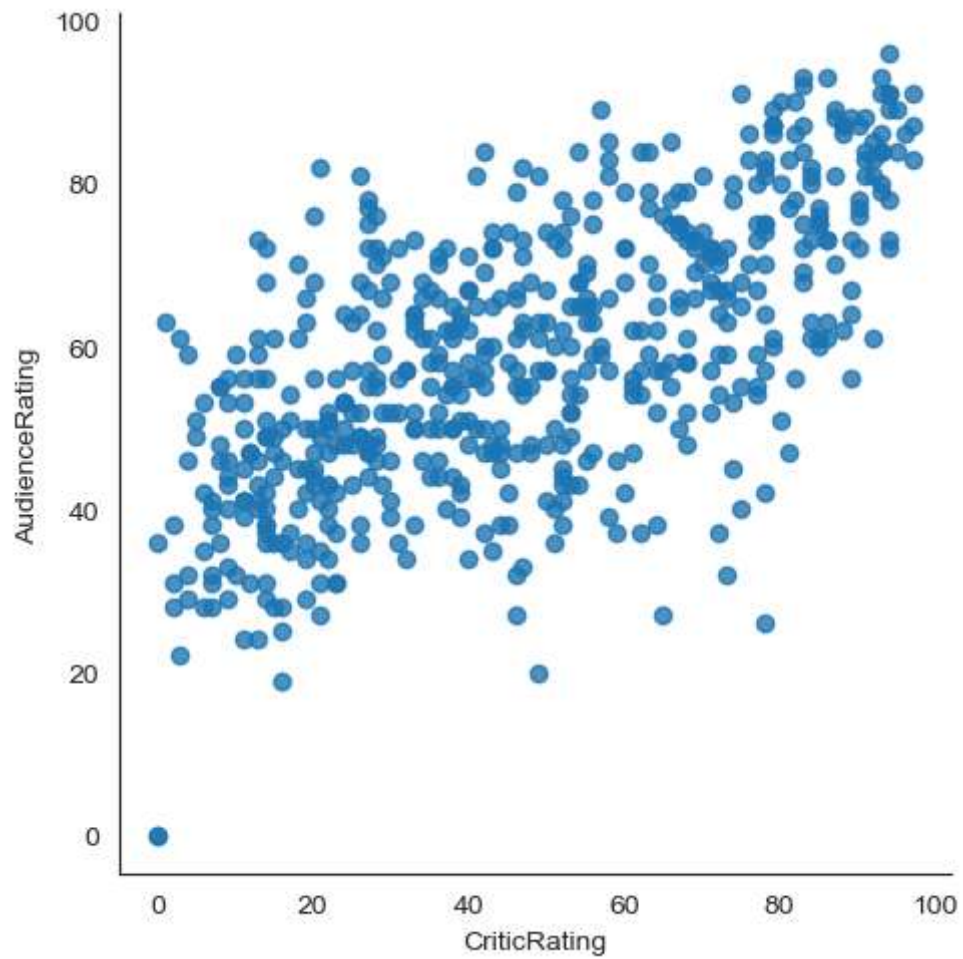


In [92]: *# if you have 100 categories you cannot copy & paste all the things*

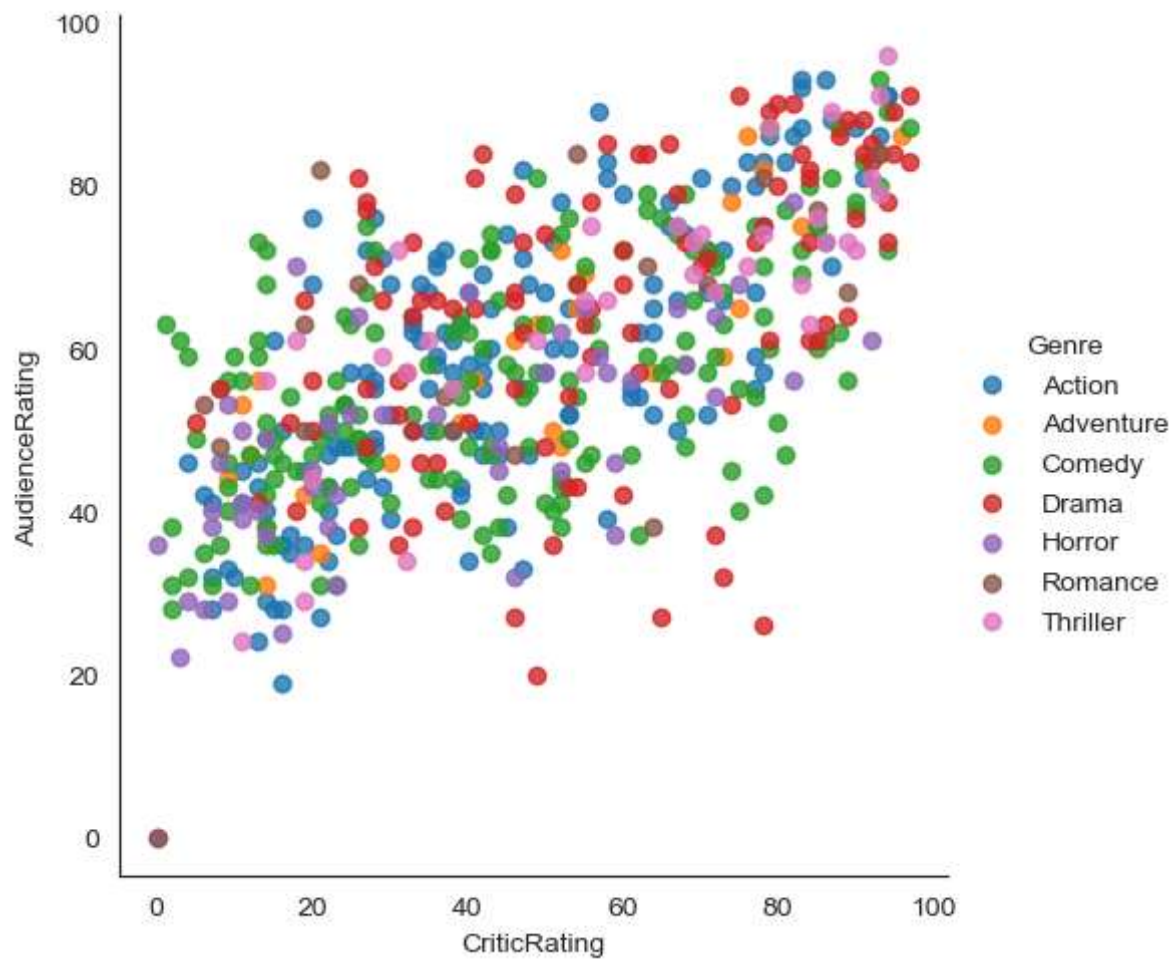
```
for gen in movies.Genre.cat.categories:
    print(gen)
```

Action
Adventure
Comedy
Drama
Horror
Romance
Thriller

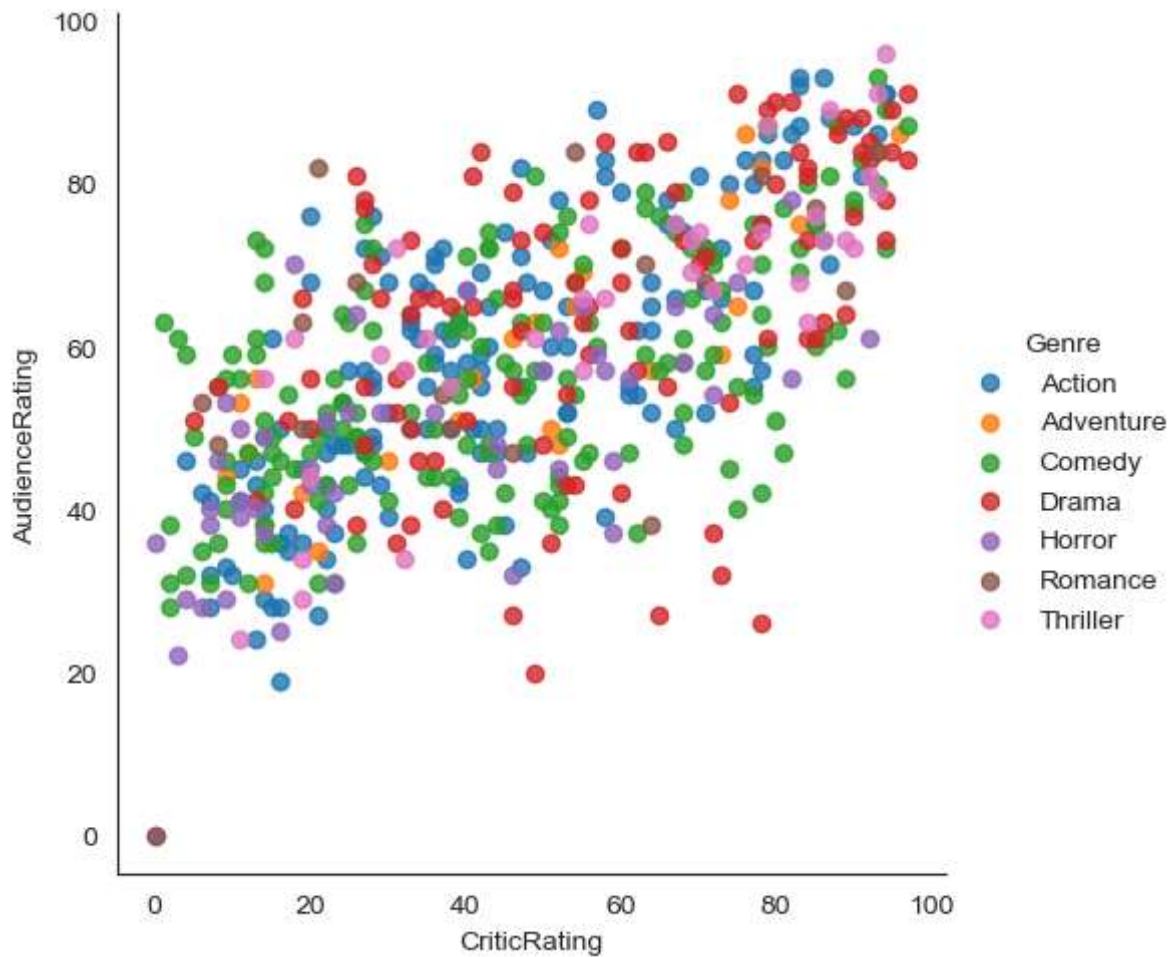
```
In [93]: vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                           fit_reg=False)
plt.show(vis1)
```



```
In [94]: vis2 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                           fit_reg=False, hue = 'Genre')
plt.show(vis2)
```



```
In [95]: vis3 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                           fit_reg=False, hue = 'Genre', aspect=1)
plt.show(vis3)
```

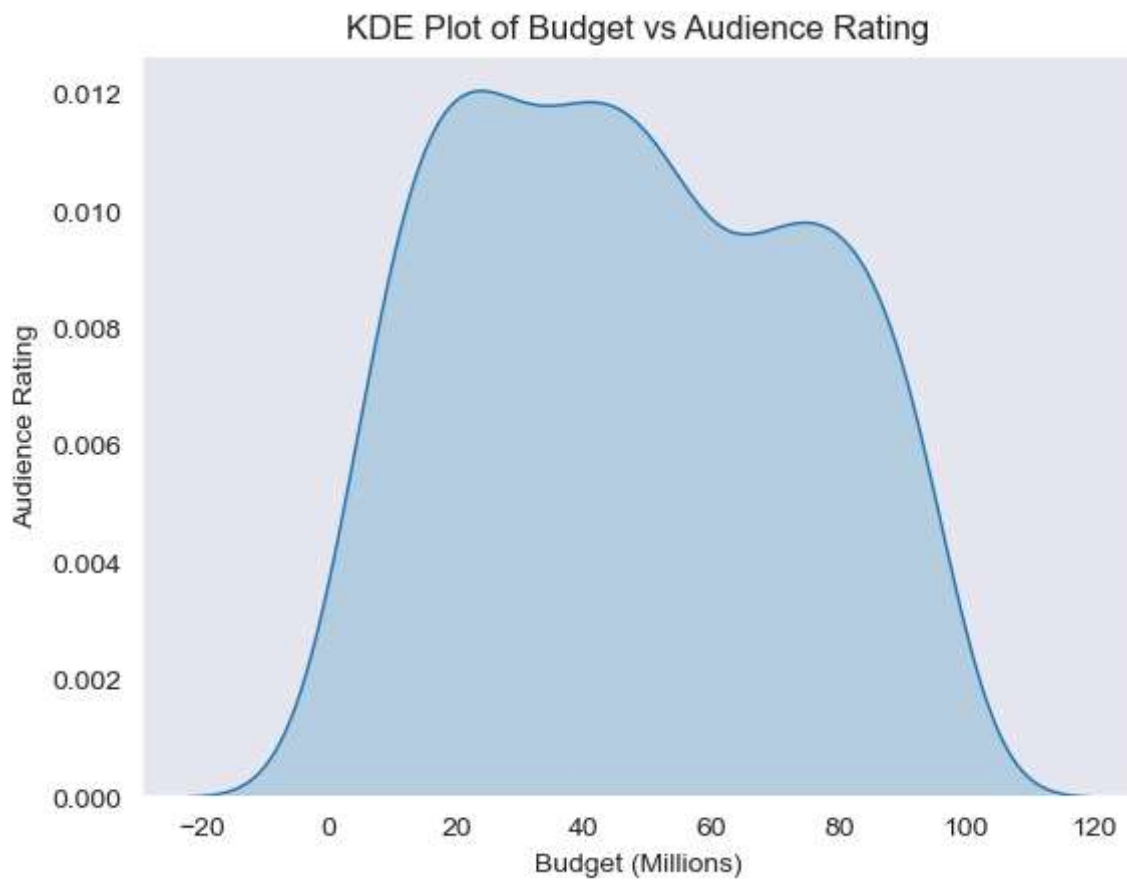


```
In [96]: # Kernal Density Estimate plot ( KDE PLOT)
# how can i visulize audience rating & critics rating . using scatterplot
```

```
In [97]: k1 = sns.kdeplot(movies.CriticRating)

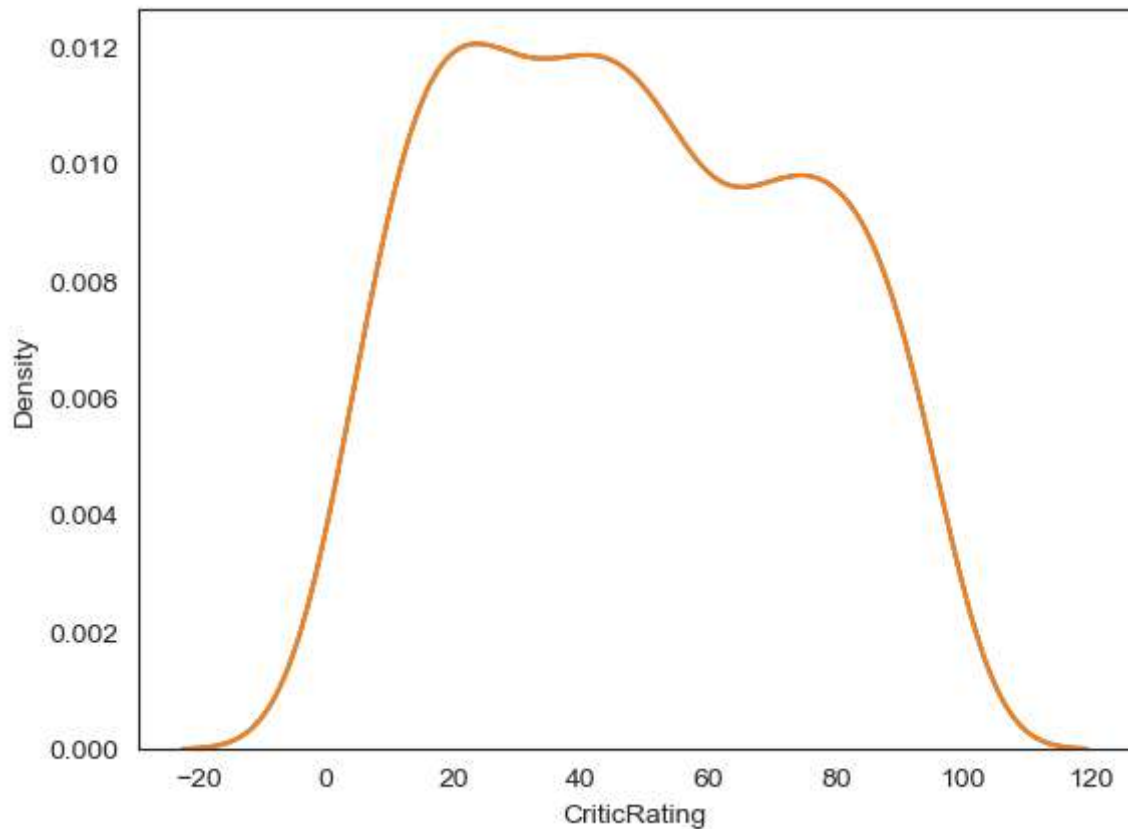
# where do u find more density and how density is distibuted across from the the ch
# center point is kernal this is calld KDE & insteade of dots it visualize like thi
# we can able to clearly see the spread at the audience ratings
```

```
In [105... k1 = sns.kdeplot(movies.CriticRating,shade = True,shade_lowest=False,cmap='Reds')
plt.title('KDE Plot of Budget vs Audience Rating')
plt.xlabel('Budget (Millions)')
plt.ylabel('Audience Rating')
plt.show()
plt.show()
```

```
In [101... k2 = sns.kdeplot(movies.CriticRating,shade_lowest=False)
display(k2)
plt.show()
```

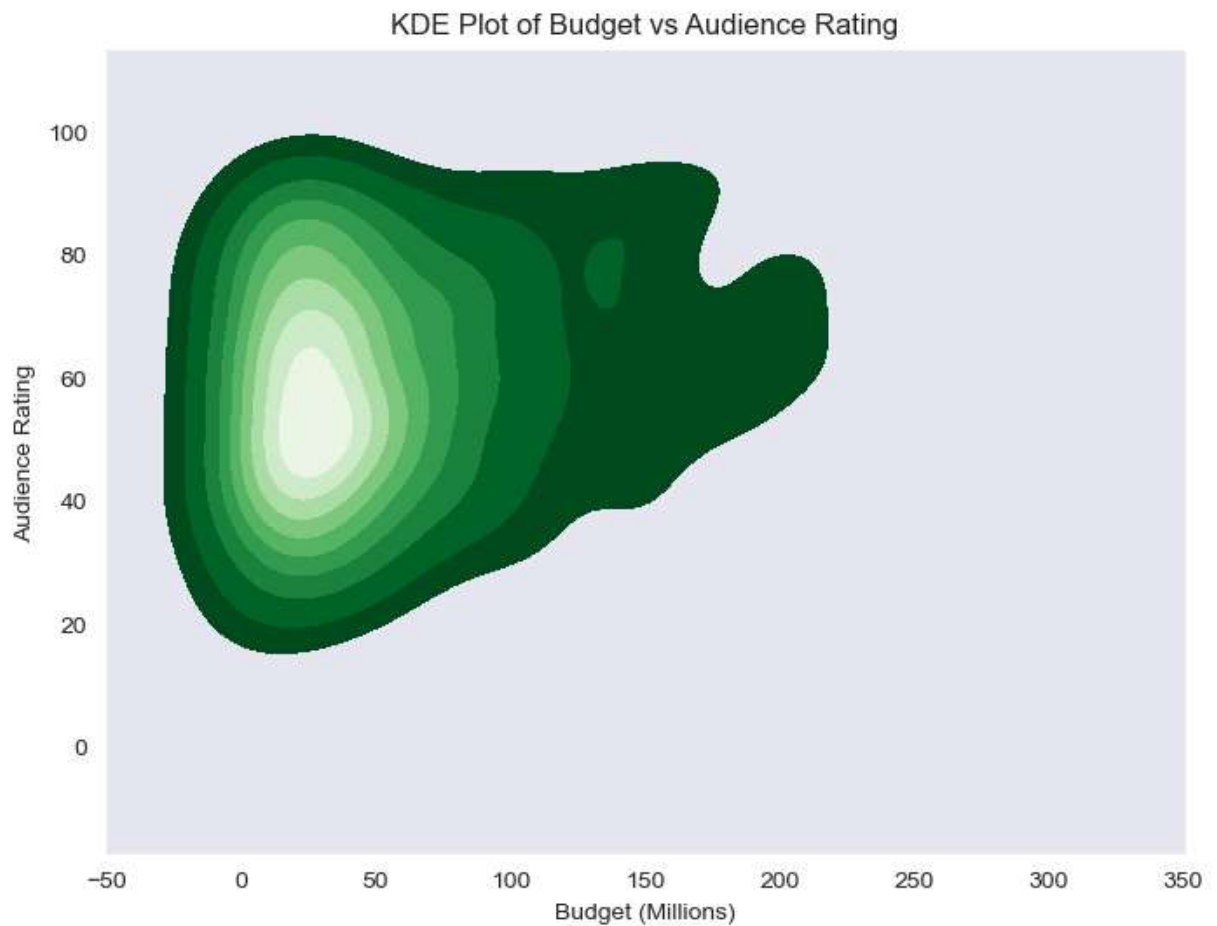
<Axes: xlabel='CriticRating', ylabel='Density'>



```
In [104... sns.set_style('dark')

plt.figure(figsize=(8,6))
k1 = sns.kdeplot(
    x=movies.BudgetMillions,
    y=movies.AudienceRating,
    cmap='Greens_r',
    fill=True,
    thresh=0.05 # controls where the fill starts (like shade_lowest used to)
)

plt.title('KDE Plot of Budget vs Audience Rating')
plt.xlabel('Budget (Millions)')
plt.ylabel('Audience Rating')
plt.show()
```

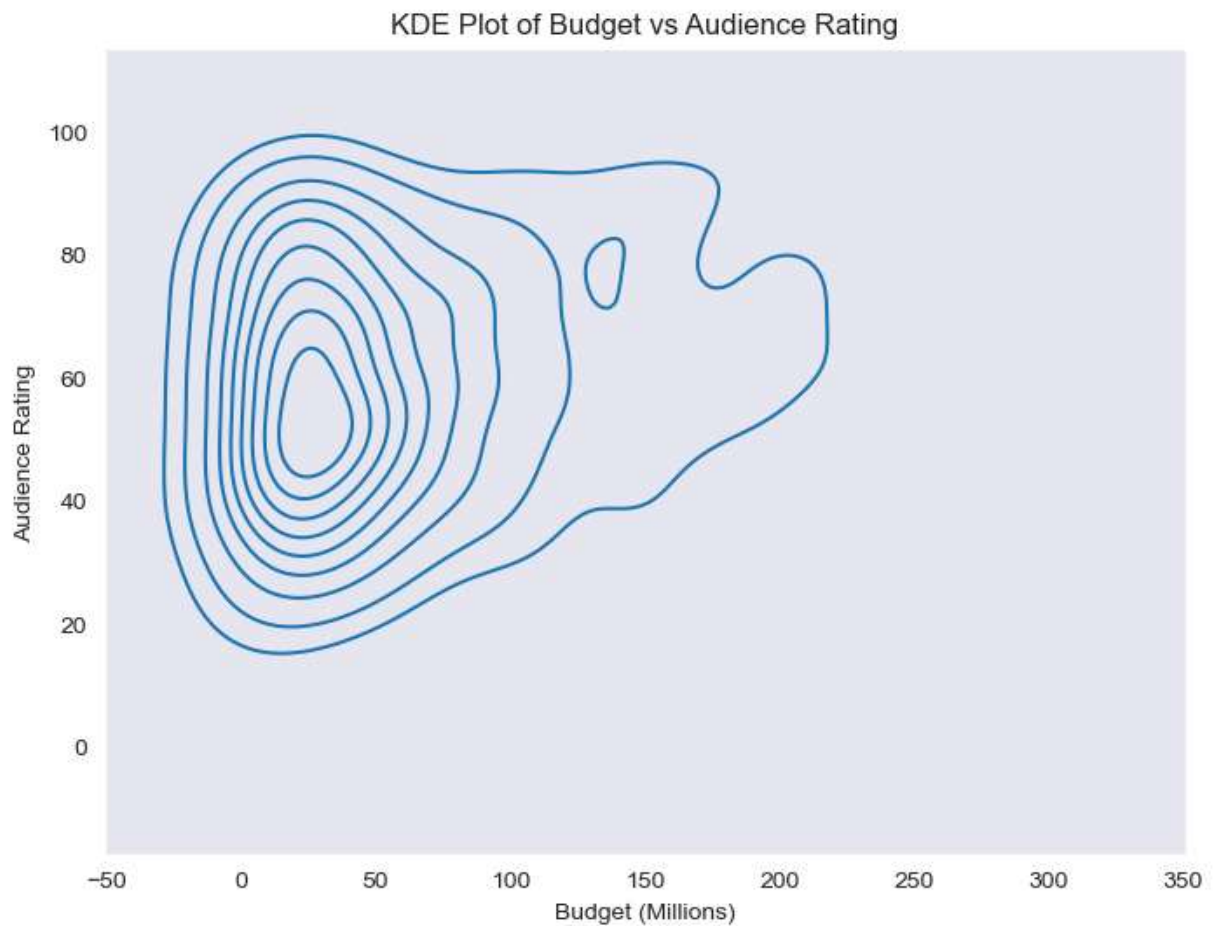


```
In [108... sns.set_style('dark')

plt.figure(figsize=(8,6)) # Optional: Better control over figure size

k1 = sns.kdeplot(
    x=movies.BudgetMillions,
    y=movies.AudienceRating
)

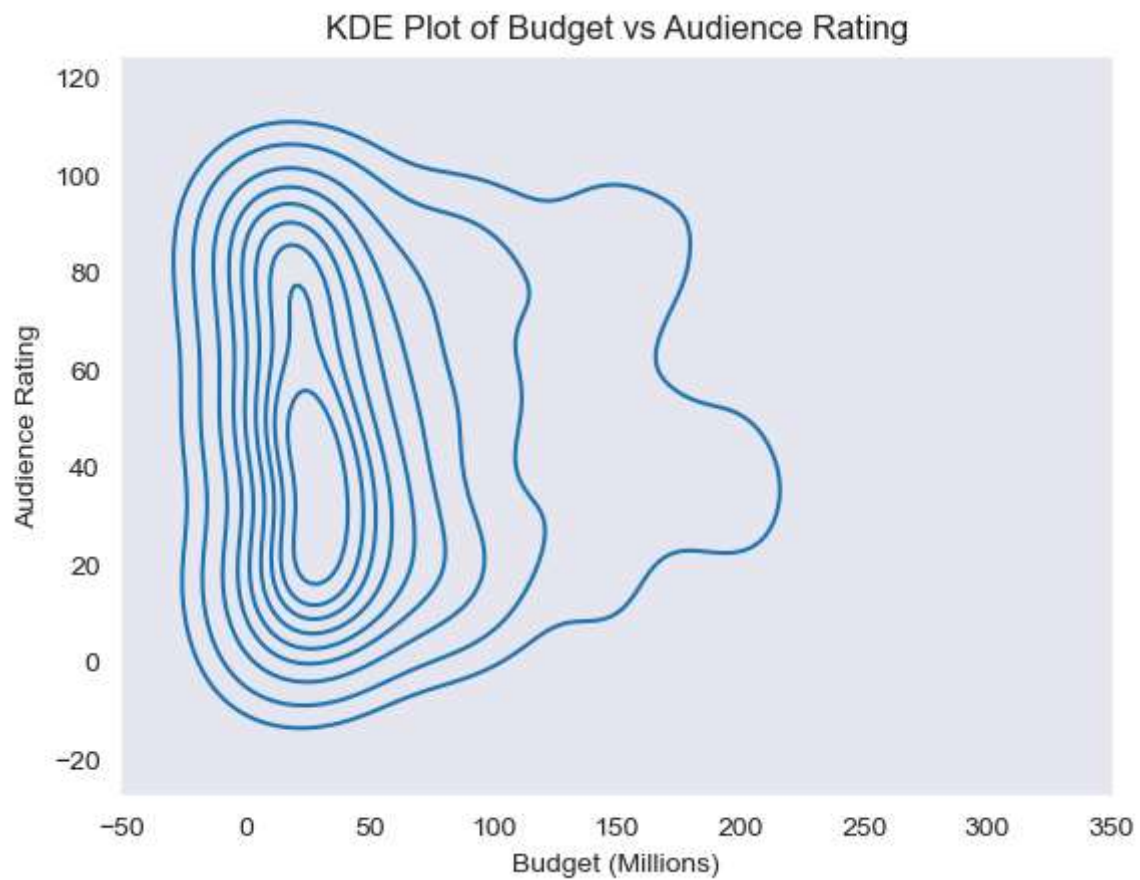
plt.title('KDE Plot of Budget vs Audience Rating')
plt.xlabel('Budget (Millions)')
plt.ylabel('Audience Rating')
plt.show()
```



In [111...

```
import seaborn as sns

k2 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating)
plt.title('KDE Plot of Budget vs Audience Rating')
plt.xlabel('Budget (Millions)')
plt.ylabel('Audience Rating')
display()
plt.show()
```

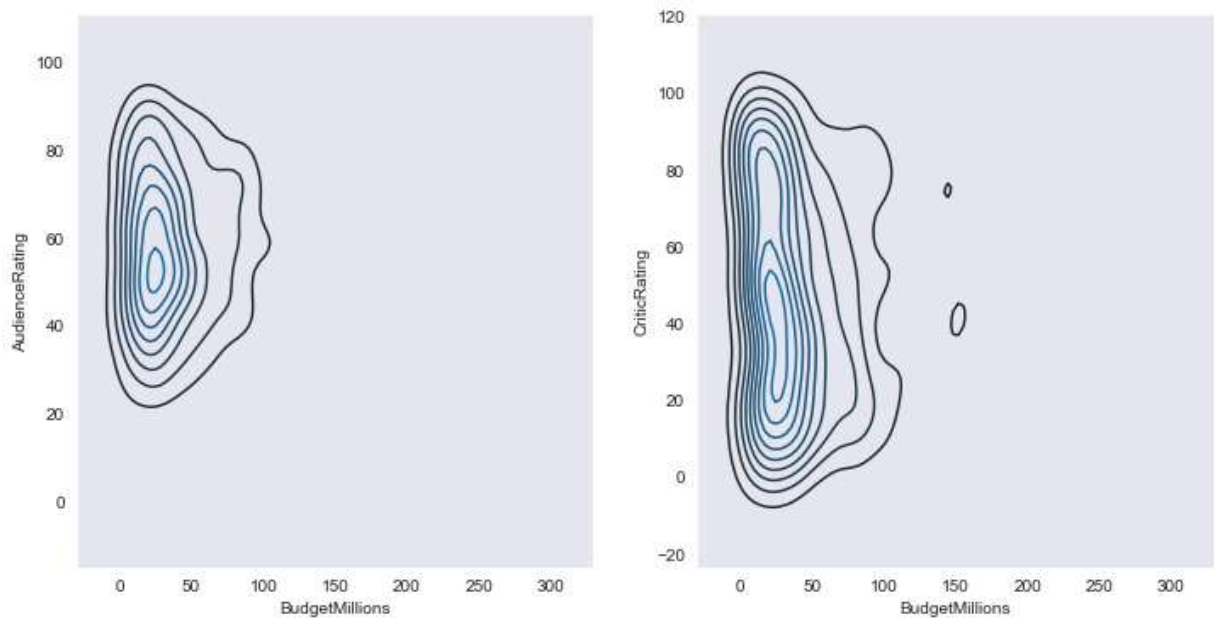


```
In [112... #subplots

f, ax = plt.subplots(1,2, figsize =(12,6))
#f, ax = plt.subplots(3,3, figsize =(12,6))
```

```
In [53]: f, axes = plt.subplots(1,2, figsize =(12,6))

k1 = sns.kdeplot(movies.BudgetMillions,movies.AudienceRating,ax=axes[0])
k2 = sns.kdeplot(movies.BudgetMillions,movies.CriticRating,ax = axes[1])
```

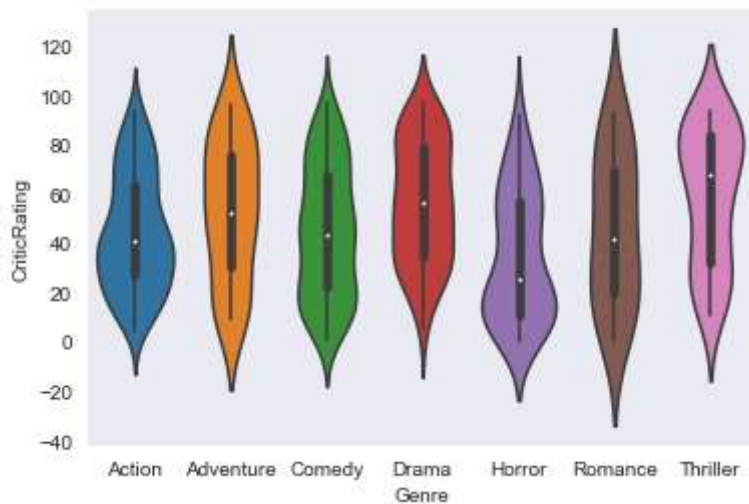


In [115... `#Box plots -`

```
w = sns.boxplot(data=movies, x='Genre', y = 'CriticRating')
```

In [56]: `#violin plot`

```
z = sns.violinplot(data=movies, x='Genre', y = 'CriticRating')
```



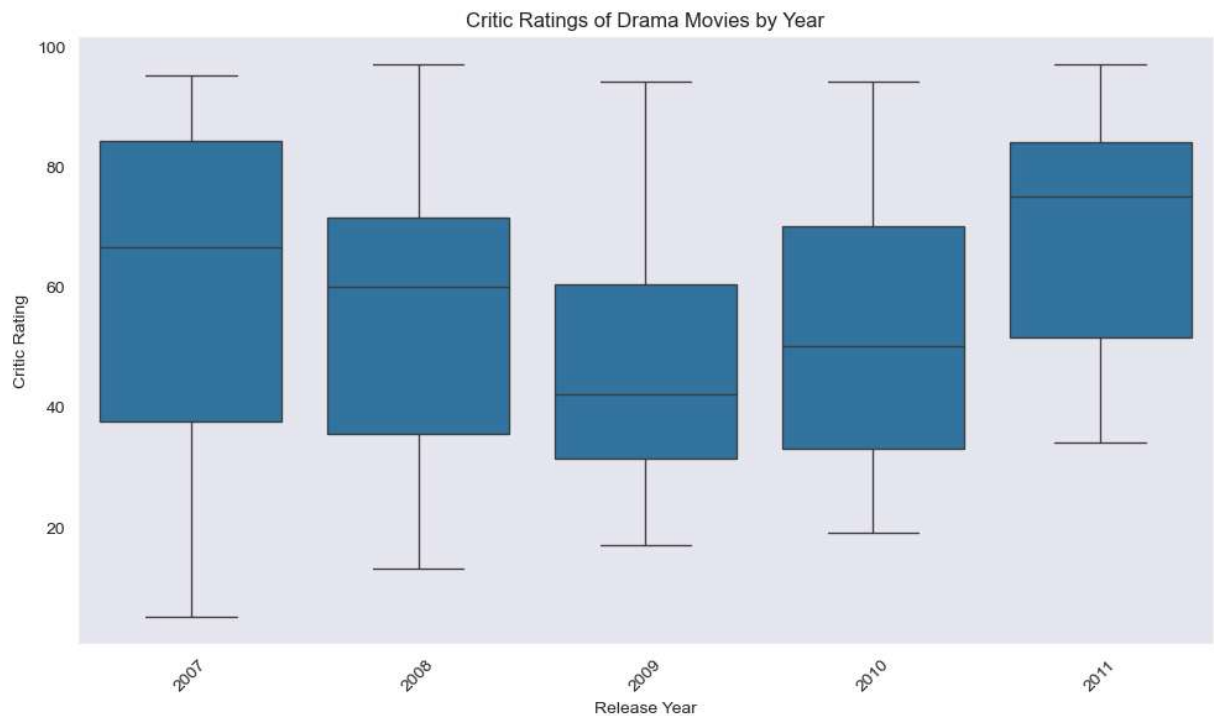
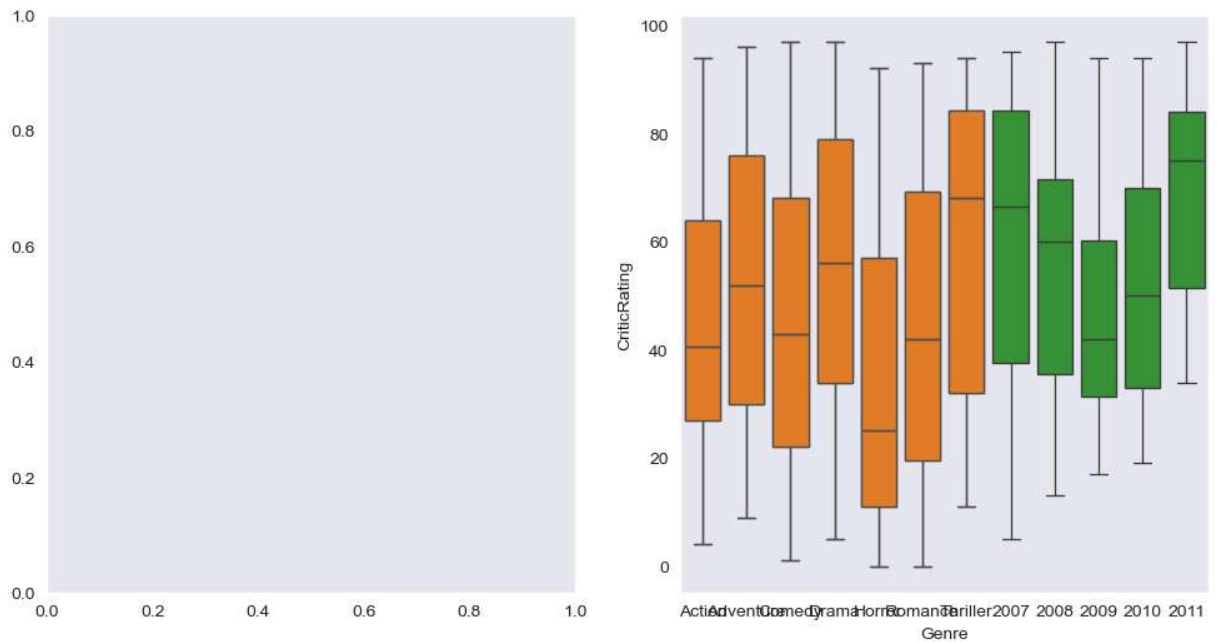
In [116... `w1 = sns.boxplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating')`

In [117... `# Filtered boxplot: Drama movies only`
`plt.figure(figsize=(10,6)) # Optional: adjust size for readability`

```
w1 = sns.boxplot(
    data=movies[movies.Genre == 'Drama'],
    x='Year',
    y='CriticRating'
)

plt.title('Critic Ratings of Drama Movies by Year')
```

```
plt.xticks(rotation=45) # Rotate x-axis labels if there are many years
plt.ylabel('Critic Rating')
plt.xlabel('Release Year')
plt.tight_layout()
plt.show()
```



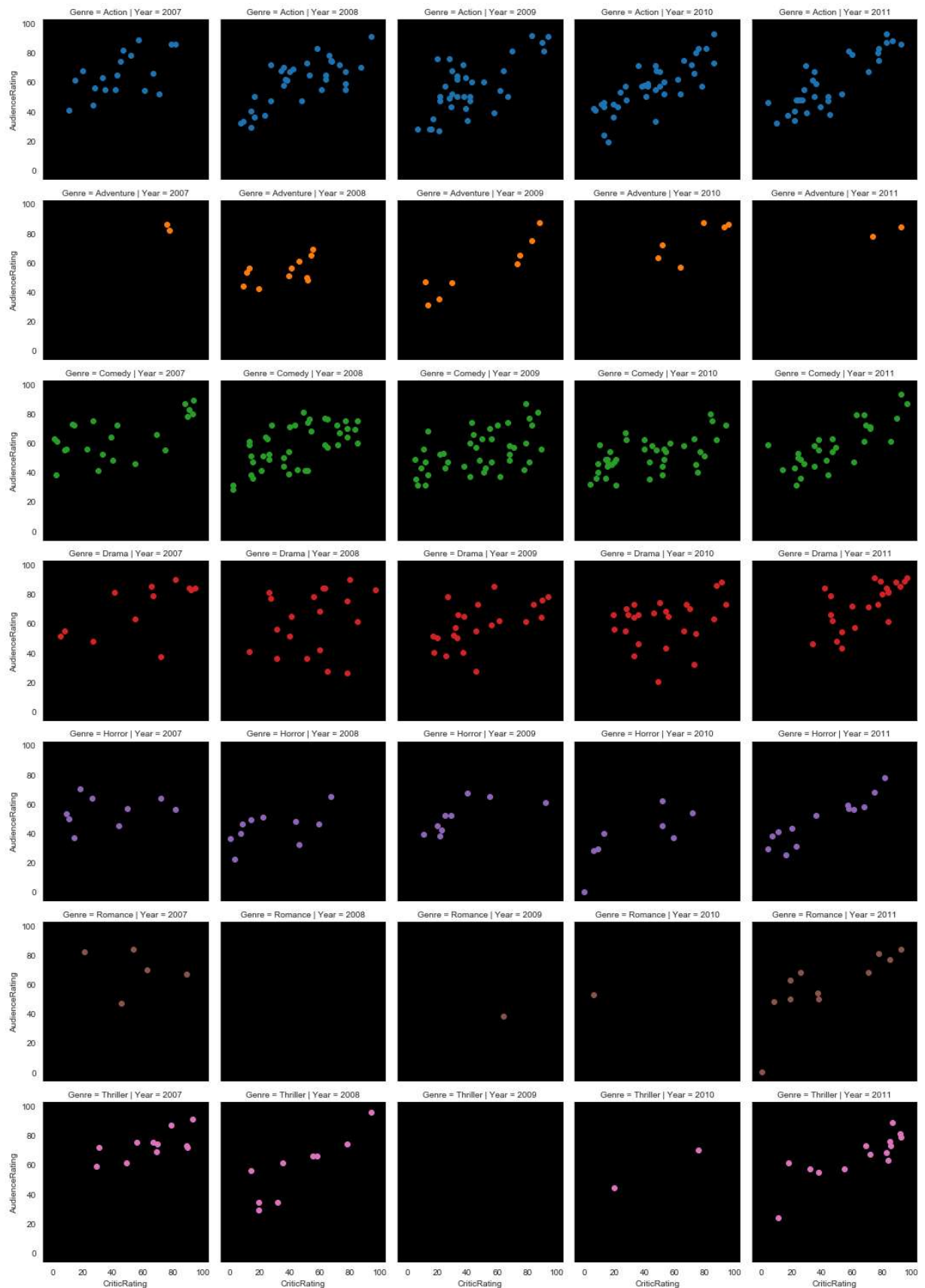
```
In [118... # Createing a Facet grid
```

```
In [119... g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre') #kind of subp
```

```
In [120... plt.scatter(movies.CriticRating,movies.AudienceRating)
```

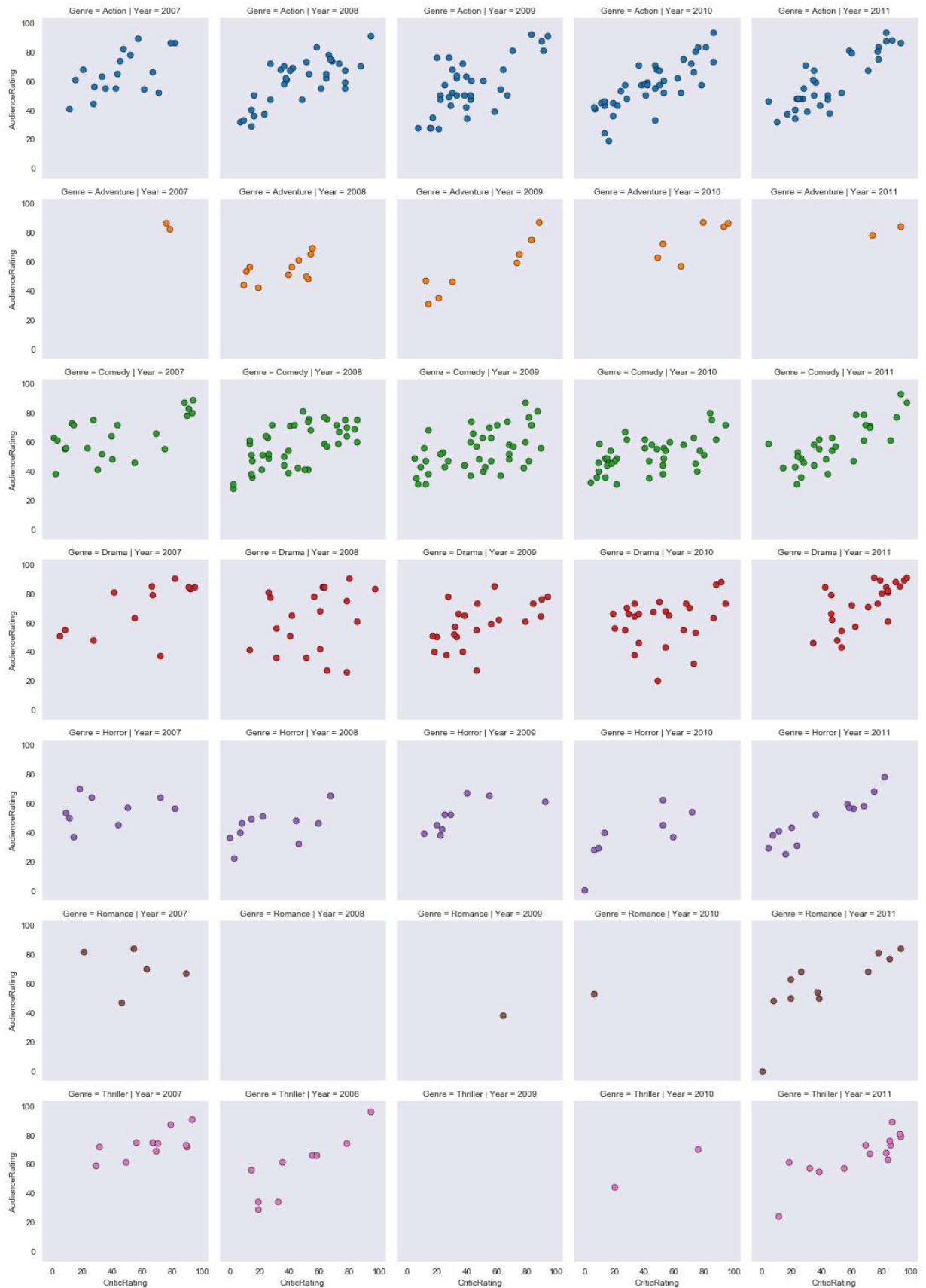
```
Out[120... <matplotlib.collections.PathCollection at 0x16562b64a50>
```

```
In [72]: g = sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating' ) #scatterplots are mapped
```




```
In [121]: # you can populated any type of chat.  
  
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')  
g = g.map(plt.hist, 'BudgetMillions') #scatterplots are mapped in facetgrid
```

```
In [64]: #  
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')  
kws = dict(s=50, linewidth=0.5,edgecolor='black')  
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating',**kws ) #scatterplots are m
```



In [126...

*# Note: Python itself isn't a vectorized language, but libraries like NumPy and pandas
 # This dashboard demonstrates multiple plots in a grid layout.*

```
sns.set_style('darkgrid')
```

```
# Create a 2x2 subplot layout
f, axes = plt.subplots(2, 2, figsize=(15, 15))

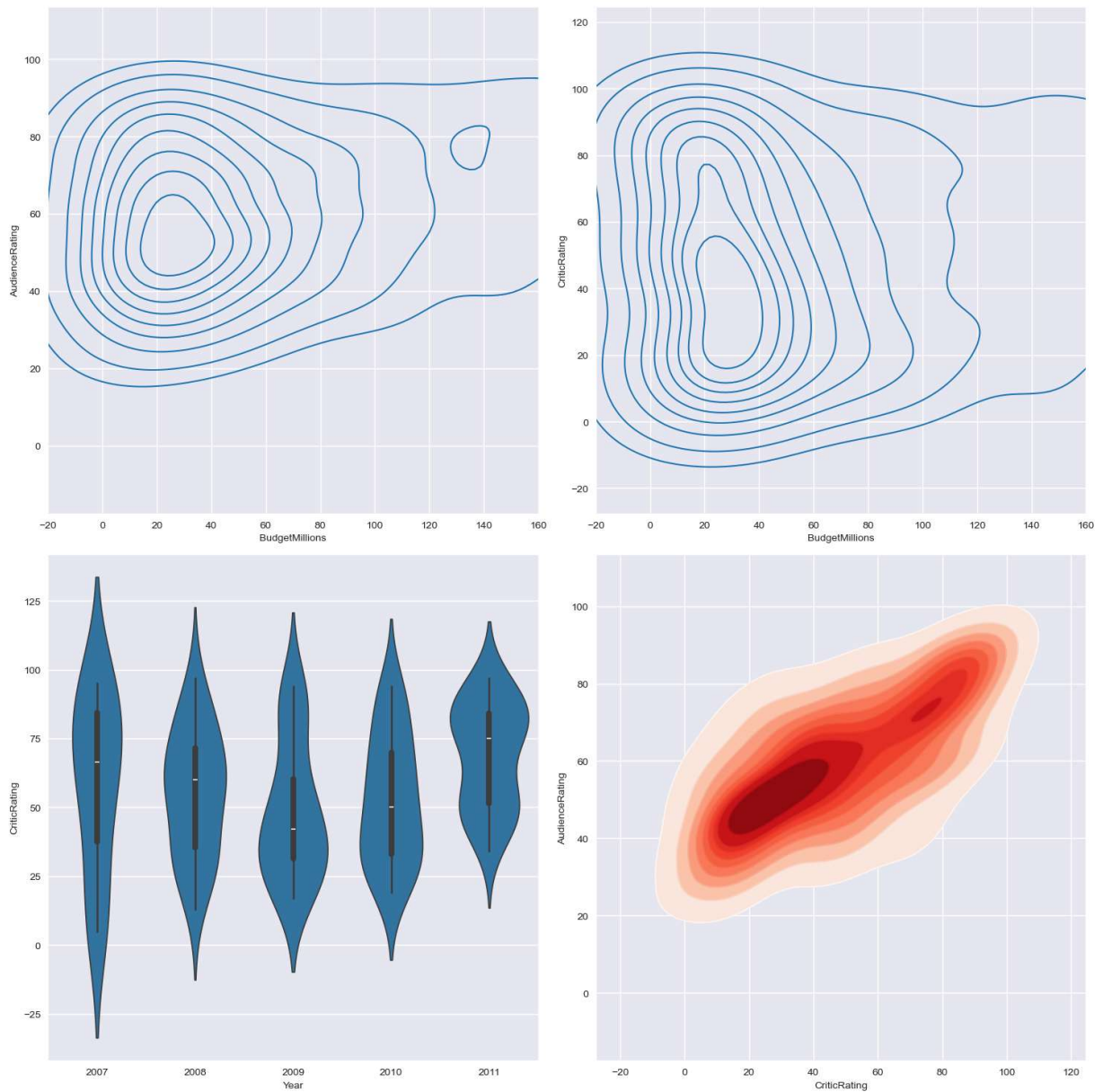
# KDE Plot: Budget vs Audience Rating
sns.kdeplot(
    x=movies.BudgetMillions,
    y=movies.AudienceRating,
    ax=axes[0, 0]
).set(xlim=(-20, 160))

# KDE Plot: Budget vs Critic Rating
sns.kdeplot(
    x=movies.BudgetMillions,
    y=movies.CriticRating,
    ax=axes[0, 1]
).set(xlim=(-20, 160))

# Violin Plot: Critic Ratings for Drama Movies by Year
sns.violinplot(
    data=movies[movies.Genre == 'Drama'],
    x='Year',
    y='CriticRating',
    ax=axes[1, 0]
)

# Bivariate KDE Plot: Critic vs Audience Rating (filled + contour)
sns.kdeplot(
    x=movies.CriticRating,
    y=movies.AudienceRating,
    fill=True,
    cmap='Reds',
    thresh=0.05,
    ax=axes[1, 1]
)
sns.kdeplot(
    x=movies.CriticRating,
    y=movies.AudienceRating,
    cmap='Reds',
    ax=axes[1, 1]
)

# Display the dashboard
plt.tight_layout()
plt.show()
```



In [129...

```
import seaborn as sns
import matplotlib.pyplot as plt

# Set style with black background for axes
sns.set_style('dark', {'axes.facecolor': 'black'})

# Create subplot grid
f, axes = plt.subplots(2, 2, figsize=(15, 15))

# === Plot [0,0]: KDE Plot - Budget vs Audience Rating ===
sns.kdeplot(
    data=movies,
    x='BudgetMillions',
    y='AudienceRating',
    fill=True,
    cmap='inferno',
    thresh=0.05,
    ax=axes[0, 0]
)
```

```

sns.kdeplot(
    data=movies,
    x='BudgetMillions',
    y='AudienceRating',
    cmap='cool',
    ax=axes[0, 0]
)

# === Plot [0,1]: KDE Plot - Budget vs Critic Rating ===
sns.kdeplot(
    data=movies,
    x='BudgetMillions',
    y='CriticRating',
    fill=True,
    cmap='inferno',
    thresh=0.05,
    ax=axes[0, 1]
)
sns.kdeplot(
    data=movies,
    x='BudgetMillions',
    y='CriticRating',
    cmap='cool',
    ax=axes[0, 1]
)

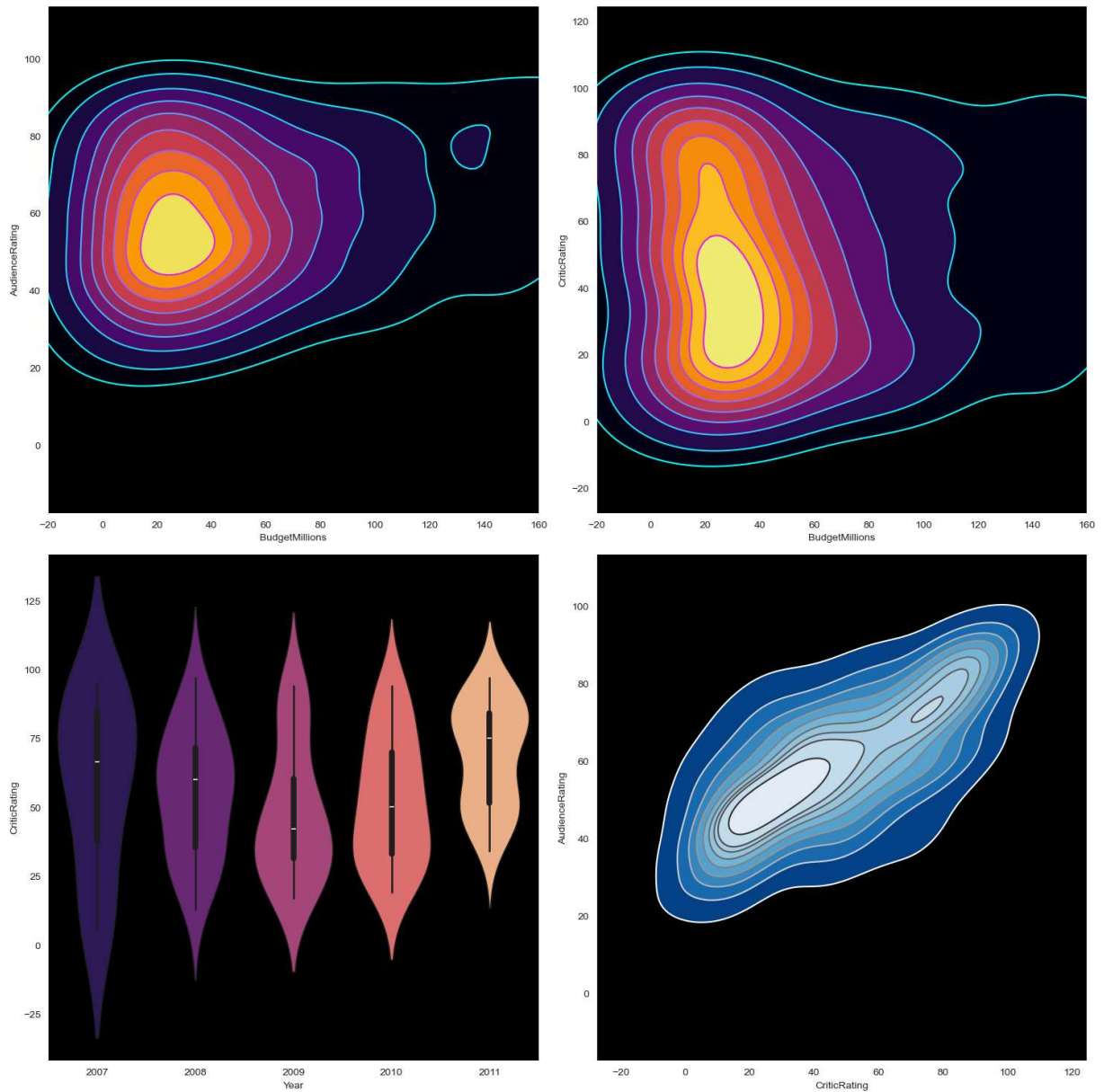
# === Plot [1,0]: Violin Plot - Critic Rating of Dramas by Year ===
sns.violinplot(
    data=movies[movies.Genre == 'Drama'],
    x='Year',
    y='CriticRating',
    palette='magma',
    ax=axes[1, 0]
)

# === Plot [1,1]: KDE Plot - Critic vs Audience Rating ===
sns.kdeplot(
    data=movies,
    x='CriticRating',
    y='AudienceRating',
    fill=True,
    cmap='Blues_r',
    thresh=0.05,
    ax=axes[1, 1]
)
sns.kdeplot(
    data=movies,
    x='CriticRating',
    y='AudienceRating',
    cmap='gist_gray_r',
    ax=axes[1, 1]
)

# Set axis Limits where needed
axes[0, 0].set_xlim(-20, 160)
axes[0, 1].set_xlim(-20, 160)

```

```
# Display the dashboard
plt.tight_layout()
plt.show()
```



Final discussion what we learn so far - 1> category datatype in python 2> jointplots 3> histogram 4> stacked histograms 5> Kde plot 6> subplot 7> violin plots 8> Faceted grid 9> Building dashboards

In [130... `# eda is completed`